VitalFlow V1 Clinical Domain — Design & Scaffold

Status: design proposal + TypeScript scaffold. The DB already has most of this live (migrations 0003 / 0004 / 0005 / 0006). This document audits what exists, names the two new entities (`DiagnosisAssignment`, richer `ClinicalDocument`), and fills in the TypeScript layer that the UI slices have been using ad-hoc.

The UI slices shipped so far are the ground truth for patient / appointment / encounter / note flow — see:

- apps/web/src/app/(app)/patients
- apps/web/src/app/(app)/appointments
- apps/web/src/app/(app)/encounters

---

1. Scope & principles

- **Multi-tenant**: every row carries `tenant_id`. Cross-tenant reads blocked by RLS. Reinforced by `has_permission()` predicates.
- **RBAC-compatible**: all write endpoints require module-level permissions from docs/permissions-matrix.md. Seed map:
  - `patient_records:*` → Patient, PatientInsurance, ClinicalDocument (patient-scoped)
  - `encounters:*`, `notes:*`, `clinical_lists:*` → Encounter, ClinicalNote, DiagnosisAssignment
  - `appointments:*` → Appointment
- **Audit-friendly**: every entity sits on a table with an `audit.log_change()` trigger. Semantic events (`note.signed`, `note.amended`, `diagnosis.assigned`) emitted via @vitalflow/auth/audit.
- **Versioning** is first-class for notes (regulatory) and optional for other entities (insurance changes, patient identity).
- **FHIR-compatible shapes** — field names and enums chosen so a FHIR resource (R4) adapter is a pure projection, not a remodel. We **do not** store FHIR JSON natively; we store normalized rows.
- **API-first** — all writes go through Server Actions (Next.js) or Route Handlers; no direct table writes from the client. RLS is the backstop.
- **Production TypeScript**: Zod schemas co-located with brand types; no `any`, no runtime validation holes at trust boundaries.

---

2. Entities

2.1 Patient

DB: `public.patients` + `public.patient_contacts`

Schema fields

Field	Type	Notes
`id`	uuid PK	
`tenant_id`	uuid	RLS anchor
`mrn`	text (unique within tenant)	Medical record number
`given_name` , `family_name`	text	Required
`preferred_name`	text?	Used in UI when set
`date_of_birth`	date	Required
`sex_at_birth`	enum `sex_at_birth`	male / female / intersex / unknown
`gender_identity`	text?	Free text, user-self-report
`pronouns`	text?	
`preferred_language`	text?	BCP-47 (`en-US` , `es-MX`)
`deceased_at`	timestamptz?	Non-null = deceased flag
`ssn_hash` , `ssn_last4`	text?	Never store full SSN
`metadata`	jsonb	For extensions, never PHI reference data

Relationships

- `patient_contacts[]` — phones, emails, addresses (typed)
- `patient_coverages[]` → `PatientInsurance`
- `encounters[]` , `appointments[]` , `orders[]` , `problems[]` , `allergies[]` , `medications[]`
- `attachments[]` → `ClinicalDocument`

Validation rules

- `mrn` unique per tenant, regex `^[A-Za-z0-9_-]{1,64}$` . Auto-generated if blank on create (base36 slug today; proper sequence-per-tenant in a follow-up).
- `date_of_birth` not in the future, not before `1900-01-01` .
- `email` and `phone` formats validated at the `patient_contacts` layer.

Lifecycle states

- `active` (default), `deceased` (set via `deceased_at`), `merged` (`metadata.merged_into_id` set + soft-delete).

- Hard delete **never permitted** — HIPAA retention.

Invariants

- `mrn` immutable after creation.
- At most one "primary" contact per `contact_type`.
- Deceased patients can still be read for billing/audit; writes blocked except by `practice_owner`.

APIs needed (Server Action + future REST)

- `POST /api/patients` → `createPatient`
- `GET /api/patients?q=&page=` → `listPatients`
- `GET /api/patients/:id` → `getPatient`
- `PATCH /api/patients/:id` → `updatePatient`
- `POST /api/patients/:id/contacts` / `DELETE /api/patients/:id/contacts/:contactId`
- `POST /api/patients/:id/merge` → `mergePatient(intoId)` — platform-admin only, v1.5

Service methods (see services.ts)

- `PatientService.create`, `list`, `get`, `update`, `markDeceased`, `addContact`, `removeContact`, `merge`

Edge cases

- Twins born same day, same parents — require distinct MRN.
- International patient without SSN — `ssn_last4` remains null; no validation.
- Name change (marriage/legal) — update in place; the audit trigger preserves prior name in `before` jsonb.
- Merge: if two MRNs represent the same human, the "losing" record gets soft-deleted with `metadata.merged_into_id`. All FK children (appointments, encounters, etc.) are re-parented in a transaction.

Acceptance

- Create patient with all required fields → row inserted, audit event row present.
- Duplicate MRN in same tenant → 409.
- Search by partial name → returns ranked results.
- Deceased patient chart renders read-only for `physician` role.

2.2 PatientInsurance

DB: `public.patient_coverages` (exists, unused by UI yet)

Schema fields

Field	Type	Notes
`id`	uuid PK	
`tenant_id`, `patient_id`	uuid	
`payer_id`	uuid → `public.payers.id`	
`type`	enum `coverage_type`	primary / secondary / tertiary / self_pay / workers_comp / auto / other
`plan_name`	text?	
`member_id`	text	Required
`group_number`	text?	
`subscriber_name`	text?	When subscriber ≠ patient
`relationship`	text?	self / spouse / child / other
`effective_start` / `effective_end`	date?	
`copay_minor`, `deductible_minor`	int?	Minor units (cents)
`currency`	char(3)	Default USD
`active`	boolean	
`metadata`	jsonb	Eligibility payload, last-verified-at

Relationships

- `patient` → `Patient`
- `payer` → `payers` (tenant-scoped; seed with common ones or free-form)

Validation rules

- Exactly one `active=true` with `type=primary` per patient at a time (app-layer; enforce via partial unique index: `(patient_id, type) WHERE type='primary' AND active`).
- `effective_start ≤ effective_end` when both set.
- Currency ISO-4217.

Lifecycle

- `pending_verification` (metadata flag), `active`, `inactive` (active=false or past effective_end), `cancelled`.

Invariants

- Primary coverage can transition to secondary only via explicit `promoteCoverage` call (business rule — audit-logged).
- Terminated coverage (`effective_end < today`) cannot be set `active=true` without a date update.

APIs

- `POST /api/patients/:id/coverages` → `addCoverage`
- `PATCH /api/patients/:id/coverages/:coverageId`
- `POST /api/patients/:id/coverages/:coverageId/verify` → calls payer eligibility API (v2)
- `DELETE /api/patients/:id/coverages/:coverageId` → soft-inactivate (active=false, set metadata.terminated_at)

Edge cases

- Switching from Medicare to commercial mid-year — create new row, inactivate old.
- Subscriber is a minor's parent — record `subscriber_name` and `relationship='child'`.
- Self-pay — insert a `type=self_pay` row with null `payer_id` (requires migration: relax NOT NULL and add CHECK constraint — noted for follow-up).

Acceptance

- Adding a coverage auto-demotes any previous active primary.
- Eligibility stored in metadata, viewable by biller.
- Claim generation (future) picks primary first, falls back to secondary.

2.3 Appointment

DB: `public.appointments`

Schema fields

Field	Type	Notes
`id`	uuid PK	
`tenant_id` , `patient_id` , `provider_id`	uuid	
`location_id`	uuid? → `locations`	
`encounter_id`	uuid? → `encounters`	Set when "Open encounter" action runs
`start_at` , `end_at`	timestamptz	
`status`	enum `appointment_status`	scheduled / confirmed / arrived / in_progress / completed / cancelled / no_show / rescheduled
`reason` , `visit_type`	text?	`visit_type` ∈ { in_person, telehealth, phone }

`telehealth_url`	text?	Generated when visit_type=telehealth
`booked_by`	uuid?	`auth.users.id` of the scheduler/user who booked
`cancelled_at`, `cancelled_reason`		
`metadata`	jsonb	

Relationships — patient, provider, optional location, optional encounter.

Validation

- `end_at > start_at`.
- Provider double-booking: soft warning in V1, hard block via exclusion constraint in V2 (`EXCLUDE USING gist (provider_id WITH =, tstzrange(start_at,end_at) WITH &&) WHERE status NOT IN ('cancelled','no_show','rescheduled')`).

Lifecycle

```
scheduled → confirmed → arrived → in_progress → completed
       \        \          \              ↓
        ↓        ↓           ↓        (completed terminal)
     cancelled  no_show   cancelled
```

`rescheduled` is a convenience state used when a new row is inserted for the same patient — the old row becomes terminal.

Invariants

- `in_progress` only if `encounter_id` is set (enforced in app; could lift to CHECK constraint).
- Cancellation requires `cancelled_reason`.
- Cannot move from `completed` back to `scheduled`.

APIs

- `POST /api/appointments` → `createAppointment`
- `GET /api/appointments?date=&provider_id=&status=` → `listAppointments`
- `GET /api/appointments/:id` → `getAppointment`
- `PATCH /api/appointments/:id` → `updateAppointment` (date/time/reason)
- `POST /api/appointments/:id/status` → `setAppointmentStatus`
- `POST /api/appointments/:id/cancel` → `cancelAppointment(reason)`
- `POST /api/appointments/:id/open-encounter` → `openEncounter` (creates encounter, links)

Edge cases

- Telehealth URL generation failure — appointment creates but `telehealth_url` null; UI retries.
- Time-zone edge — see V1 caveat in Slice 2; day-view filter currently uses UTC.
- Patient no-show but also cancels → last-write-wins; audit log captures both states.

Acceptance

- Book → row inserted, audit event.
- `open_encounter` creates encounter + sets `encounter_id` + advances appointment status atomically (RPC in v2).

2.4 Encounter

DB: `public.encounters`

Schema fields

Field	Type	Notes
`id`	uuid PK	
`tenant_id`, `patient_id`, `provider_id`	uuid	
`class`	enum `encounter_class`	ambulatory / emergency / inpatient / virtual
`status`	enum `encounter_status`	planned / arrived / in_progress / finished / cancelled
`start_at`	timestamptz	
`end_at`	timestamptz?	Set when status=finished
`location`	text?	Free-text for now; FK to `locations` in v2
`chief_complaint`	text?	
`reason`	text?	
`metadata`	jsonb	

Relationships

- `patient` → `Patient`
- `provider` → `auth.users`
- `notes[]` → `ClinicalNote`
- `orders[]`, `vitals[]`, `attachments[]`, `diagnoses[]` → `DiagnosisAssignment`
- Optional back-pointer from `appointments.encounter_id`

Lifecycle

```
planned → arrived → in_progress → finished
   \                       ↓
    → cancelled      finished (terminal)
```

Invariants

- `end_at` required when `status=finished`.
- Encounter can only be finished if its active note is `signed` (app-layer gate — V2 raises to trigger).
- Cancelled encounters keep their rows (soft-cancel for audit); no hard delete.

APIs

- `POST /api/encounters` → `createEncounter` (rare — usually via `open-encounter` on appointment)
- `GET /api/encounters?mine=&status=` → `listEncounters`
- `GET /api/encounters/:id` → `getEncounter`
- `PATCH /api/encounters/:id` → `updateEncounter` (status, chief_complaint, reason)
- `POST /api/encounters/:id/finish` → `finishEncounter` (checks active note signed)

Edge cases

- Encounter started without an appointment (walk-in) — allowed; appointment remains null.
- Provider hand-off mid-encounter — update `provider_id` but keep original in `metadata.handoff_chain[]`.
- Finished encounter with unsigned note — UI shows "note pending sign-off" banner, prevents finishing until signed.

Acceptance

- Open-encounter creates row, links to appointment.
- Finish requires a signed active note.
- Cancelled encounter preserves row.

2.5 ClinicalNote

DB: `public.encounter_notes` (single table — see §2.6 for versioning)

Schema fields

Field	Type	Notes
`id`	uuid PK	
`tenant_id`, `encounter_id`, `patient_id`, `author_id`	uuid	

`type`	enum `note_type`	soap (default), progress, consult, discharge, addendum
`status`	enum `note_status`	draft / pending_review / signed / amended
`subjective`, `objective`, `assessment`, `plan`	text?	SOAP fields
`free_text`	text?	For non-SOAP notes
`ai_assisted`	boolean	Flag — set when an AI draft seeded the note
`ai_request_id`	uuid? → `ai_requests`	
`signed_by`	uuid?	auth.users
`signed_at`	timestamptz?	
`amended_from`	uuid? → `encounter_notes.id`	Set on amendment versions
`version`	smallint	1-indexed

Relationships

- `encounter` → `Encounter`
- `patient` → `Patient` (denormalized for RLS + search)
- `author`, `signed_by` → `auth.users`
- `signatures[]` → rows in `public.signatures` keyed by `subject_table='encounter_notes'`, `subject_id=note.id`

Validation

- Either SOAP fields OR `free_text` populated on save; both allowed.
- `status=signed` requires `signed_by`, `signed_at` both set.
- On amendment insert, `amended_from` must reference a row whose `status='signed'`.

Lifecycle

```
draft → pending_review? → signed → amended (terminal for this version;
                              ↑              next version is a new row)
                              └── only path forward from signed is amendment
```

`pending_review` is for co-sign workflows (attending countersigning a resident); v2.

Invariants

- Exactly one note per encounter with `status NOT IN ('amended')` — the "current" note. App-enforced today; future: partial unique index.
- Version numbers strictly increasing within an encounter's amendment chain.
- Signed note fields are immutable — any UPDATE that touches content while `status='signed'` is rejected (add a trigger).
- Sign operation writes both the note row update and a `public.signatures` row atomically (V2 RPC).

APIs

- `POST /api/encounters/:encounterId/notes` → `createDraftNote`
- `PATCH /api/encounters/:encounterId/notes/:noteId` → `saveNoteDraft`
- `POST /api/encounters/:encounterId/notes/:noteId/sign` → `signNote(attestation?)`
- `POST /api/encounters/:encounterId/notes/:noteId/amend` → `amendNote(reason)` (returns new draft id)
- `GET /api/encounters/:encounterId/notes/current` → returns the note with `status != 'amended'` and highest version
- `GET /api/encounters/:encounterId/notes/history` → returns all versions ordered `version DESC`

Edge cases

- Two users edit the same draft concurrently — last-write-wins today; need optimistic-lock column (`updated_at` check) before production.
- Long SOAP text — no DB length limit; UI caps at 64k chars.
- Note signed while the encounter is cancelled — allowed (people cancel after signing). App shows both.
- Multi-author notes (nurse drafts + physician signs) — author stays as nurse, `signed_by` is physician. Both in audit trail.

Acceptance

- Draft create + save round-trip.
- Sign flips status + writes signature row + content hash (already implemented).
- Amend creates new version, old version flipped to `amended`, new draft pre-fills content.

2.6 ClinicalNoteVersion — design decision

The prompt asks for "clean separation between current note and note versions." Two shapes:

Option A (current implementation): one table, chain via `amended_from`, status=`amended` marks superseded rows.

- ✅ Simpler writes, one audit trigger, no duplication.
- ✅ Matches what's shipped.
- ❌ "Current" is computed (`status != 'amended'` + highest version), not declared.
- ❌ No type-level distinction between "active" and "historic" rows.

Option B: split — `clinical_notes` (current only, one per encounter) + `clinical_note_versions` (full snapshots of every historical write).

- ✅ Constrained: `UNIQUE(encounter_id)` on the current table.
- ✅ "Current" query is a single-row read.
- ❌ Double-write on every save (current update + version insert).
- ❌ Two audit trigger points.
- ❌ Amendment flow rewires two tables; more moving parts.

Recommendation: Option A + a VIEW. Ship a `public.clinical_notes_current` view defined as:

```
create view public.clinical_notes_current as
  select distinct on (encounter_id) *
  from public.encounter_notes
  where status <> 'amended'
  order by encounter_id, version desc;
```

This gives the domain layer a "current vs. versions" distinction without the double-write tax. UI code queries the view for the workspace; audit / history queries the base table.

Versions are simply rows in `encounter_notes` with `amended_from IS NOT NULL` plus the root row.

2.7 DiagnosisAssignment — new entity

Not in DB today. `public.problems` is the patient-level problem list (long-running conditions). **DiagnosisAssignment** is the encounter-scoped billing-relevant mapping of ICD-10-CM codes to a specific visit. Needed for claims generation and clinical note finalization.

Proposed table: `public.diagnosis_assignments`

Field	Type	Notes
`id`	uuid PK	
`tenant_id` , `patient_id` , `encounter_id`	uuid	
`problem_id`	uuid? → `problems.id`	Optional link when the encounter reinforces an existing problem

`code_system`	text	Always `'icd10-cm'` in V1; future `'icd11'`, `'snomed'`
`code`	text	e.g. `E11.9`
`description`	text	Denormalized display
`rank`	smallint	1 = primary, 2 = secondary, …
`pointer`	text?	CMS-1500 diagnosis pointer (A–L)
`present_on_admission`	text?	`Y/N/U/W` flag (inpatient only)
`assigned_by`	uuid	auth.users
`assigned_at`	timestamptz	Default `now()`
`removed_at`	timestamptz?	Soft-remove flag

Relationships

- `encounter` → `Encounter`
- `patient` → `Patient`
- `problem` → `problems` (optional)

Validation

- `rank` must be unique per `(encounter_id, rank)` for non-removed rows (partial unique index).
- `code` must match the ICD-10-CM regex `^[A-Z][0-9]{2}(\\.[0-9A-Z]{1,4})?$` (loose — exhaustive validation is a lookup against an ICD dictionary, v2).
- `present_on_admission` only valid for inpatient encounter_class.

Lifecycle

- `assigned` (default) → `removed` (soft). Removed assignments don't hit claims.

Invariants

- Exactly one `rank=1` per encounter (partial unique index).
- `assigned_by` must have `clinical_lists:update` permission.
- Signed note cannot have diagnoses added/removed without an amendment reason logged (V2).

APIs

- `POST /api/encounters/:id/diagnoses` → `assignDiagnosis({ code, rank, problem_id? })`
- `GET /api/encounters/:id/diagnoses` → `listDiagnoses`
- `PATCH /api/encounters/:id/diagnoses/:daId` → `updateDiagnosis` (rank, pointer)
- `DELETE /api/encounters/:id/diagnoses/:daId` → `removeDiagnosis` (soft)

- `POST /api/encounters/:id/diagnoses/reorder` → bulk-set rank array

Service methods — see `DiagnosisService` in services.ts.

Edge cases

- Same ICD-10 code assigned twice to one encounter — allowed but warned (e.g. different modifiers).
- Promoting a diagnosis to the problem list — `promoteToProblem(daId)` creates a `problems` row and sets `diagnosis_assignments.problem_id`.
- Billing pulls diagnoses in rank order; rank gaps (1,3 with no 2) are auto-compacted on save.

Acceptance

- Assign ICD-10 `E11.9` → row inserted, present in `/encounters/:id/diagnoses` list.
- Reorder ranks → single transaction, no duplicates.
- Removed diagnoses excluded from claim preview.

2.8 ClinicalDocument — extend `attachments`

`public.attachments` **exists as generic file storage.** Rather than a new table, we'd extend it with the structured fields needed for a true clinical document.

Proposed additions to `public.attachments`

Field	Type	Notes
`kind`	text (new)	`note_pdf`, `lab_report`, `imaging_report`, `discharge_summary`, `intake_form`, `consent`, `other`
`signed_by`	uuid? (new)	auth.users
`signed_at`	timestamptz? (new)	
`effective_date`	date? (new)	Clinical effective date (may differ from upload date)
`source`	text? (new)	`upload`, `generated`, `ehr_import`, `fax`

The existing columns (`storage_bucket`, `storage_path`, `mime_type`, `sha256`, `label`, `category`, `metadata`) stay as-is. A plain file upload skips the new fields; a structured clinical doc populates them.

Relationships — same as today: optional `patient_id`, optional `encounter_id`.

Validation

- `kind='note_pdf'` requires `encounter_id` + `signed_by` + `signed_at`.

- `kind='consent'` requires `signed_by` + `signed_at` + `effective_date`.
- `sha256` computed on upload; mismatches between storage and the column value are a drift alert.

Lifecycle

- `uploaded` → `signed` (when `signed_by` set) → `superseded` (when a newer document of same `(patient_id, kind, encounter_id)` is uploaded — optional).
- Soft-delete via `deleted_at` (already exists).

Invariants

- Storage path is append-only; rewrites require a new row.
- `sha256` present for any `kind != 'other'`.

APIs

- `POST /api/attachments` → `uploadDocument({ file, kind, patient_id, encounter_id?, effective_date? })`
- `GET /api/attachments?patient_id=&encounter_id=&kind=` → `listDocuments`
- `GET /api/attachments/:id/download` → signed URL for short-lived download
- `POST /api/attachments/:id/sign` → mark signed (separate from upload)
- `DELETE /api/attachments/:id` → soft-delete (audit-logged)

Edge cases

- Large imaging (DICOM) — kept out of the main bucket; store a ref in `metadata.dicom_study_uid`.
- Consent superseded — previous row stays signed, new row is the current one.
- Unsigned PDF uploaded but later signed externally — allowed; `signed_by` filled on a later PATCH.

Acceptance

- Upload a PDF, mark kind=`note_pdf`, sign → row created, signed, audit event.
- Download returns a signed URL expiring in 5 min.

---

3. Cross-entity invariants

- **Tenant consistency**: every FK must target a row with the same `tenant_id`. Enforced by app code + RLS; future CHECK constraints can co-locate both sides.
- **Soft-delete propagation**: a soft-deleted patient does NOT cascade-soft-delete their encounters/notes. Audit trail stays fully intact.

- **Signed-content immutability**: any write path (UI, API, direct SQL) that modifies a `signed` note / signature / document is blocked except via explicit amendment RPC.
- **Impersonation**: all write endpoints respect `IMPERSONATION_BLOCKED` strip (see permissions-matrix.md §4.1). A super-admin impersonating cannot `sign`, `amend`, or `export`.

---

4. API route design

All routes are `/api/v1/...` in the future full REST surface. Today they're Server Actions inside the Next app; the shapes are identical.

Convention:

- `POST` to create, `GET` to read, `PATCH` to partial-update, `DELETE` to soft-delete.
- All responses are `{ data, error?, meta? }` JSON.
- Idempotency via `Idempotency-Key` header for `POST` (v2).

Route sketch

```
/api/v1/patients
  POST   /                 createPatient
  GET    /                 listPatients        ?q= &page= &limit=
  GET    /:id              getPatient
  PATCH  /:id              updatePatient
  POST   /:id/contacts     addPatientContact
  DELETE /:id/contacts/:cid removePatientContact
  POST   /:id/coverages    addCoverage
  PATCH  /:id/coverages/:covId  updateCoverage
  DELETE /:id/coverages/:covId  removeCoverage

/api/v1/appointments
  POST   /                 createAppointment
  GET    /                 listAppointments    ?date= &provider_id= &status=
  GET    /:id              getAppointment
  PATCH  /:id              updateAppointment
  POST   /:id/status       setAppointmentStatus   { status }
  POST   /:id/cancel       cancelAppointment      { reason }
  POST   /:id/open-encounter  openEncounterFromAppointment → { encounter_id }

/api/v1/encounters
  POST   /                 createEncounter
  GET    /                 listEncounters      ?mine= &status=
  GET    /:id              getEncounter
  PATCH  /:id              updateEncounter
  POST   /:id/finish       finishEncounter

  GET    /:id/notes/current  getCurrentNote
  GET    /:id/notes/history  getNoteHistory
  POST   /:id/notes          createDraftNote
  PATCH  /:id/notes/:noteId  saveNoteDraft
  POST   /:id/notes/:noteId/sign   signNote   { attestation? }
  POST   /:id/notes/:noteId/amend  amendNote  { reason } → { new_note_id }

  GET    /:id/diagnoses      listDiagnoses
  POST   /:id/diagnoses      assignDiagnosis { code, rank, problem_id? }
  PATCH  /:id/diagnoses/:daId updateDiagnosis
  DELETE /:id/diagnoses/:daId removeDiagnosis
  POST   /:id/diagnoses/reorder reorderDiagnoses { ranks: [id, ...] }

/api/v1/attachments
  POST   /                 uploadDocument
  GET    /                 listDocuments     ?patient_id= &encounter_id= &kind=
  GET    /:id/download     downloadDocument  → signed URL
  POST   /:id/sign         signDocument
  DELETE /:id              removeDocument
```

---

5. TypeScript scaffold

New TypeScript types live in:

- `packages/types/src/clinical/patient.ts`
- `packages/types/src/clinical/appointment.ts`
- `packages/types/src/clinical/encounter.ts`
- `packages/types/src/clinical/note.ts`
- `packages/types/src/clinical/diagnosis.ts`
- `packages/types/src/clinical/document.ts`
- `packages/types/src/clinical/services.ts`

Each file exports:

- Zod schemas for the entity (full + `Create`, `Update`, `List` variants).
- Brand types for IDs.
- Enum types.
- Re-exports from `packages/types/src/clinical/index.ts`.

Service interfaces (no impls) live in `services.ts`. Repository implementations will use `@vitalflow/auth/server`'s Supabase client and get generated alongside API route handlers in a follow-up PR.

---

6. Seed data

Minimum seed for end-to-end testing (run after the bootstrap user + tenant are in place):

```sql
-- 1 payer, 2 patients, 1 appointment, 1 encounter, 1 signed note, 2 diagnoses

insert into public.payers (tenant_id, name, payer_code, active)
values ((select id from public.tenants where slug='demo'),
        'BlueCross BlueShield', 'BCBS', true)
on conflict do nothing;

-- Patients (use valid sex_at_birth values)
insert into public.patients (tenant_id, mrn, given_name, family_name, date_of_birth, sex_at_
values
  ((select id from public.tenants where slug='demo'), 'MRN-SEED-001', 'Alex', 'Morgan',   '1
  ((select id from public.tenants where slug='demo'), 'MRN-SEED-002', 'Priya', 'Shah',    '1
on conflict do nothing;

-- Coverage for patient 1
insert into public.patient_coverages
  (tenant_id, patient_id, payer_id, type, plan_name, member_id, effective_start, active)
select t.id, p.id, py.id, 'primary', 'PPO Gold', 'X1234567', current_date - interval '2 year
from public.tenants t
join public.patients p on p.tenant_id = t.id and p.mrn = 'MRN-SEED-001'
join public.payers py on py.tenant_id = t.id and py.payer_code = 'BCBS'
where t.slug = 'demo'
on conflict do nothing;

-- (Appointment, encounter, note, diagnosis assignment seeds require an existing
-- provider user id — generate via the UI once a tenant member is present, or
-- parametrize with supabase/seed.sql and the bootstrap SQL snippet.)
```

For repeatable dev seeds, fold these into `supabase/seed.sql` after wrapping them in `where not exists (...)` guards so the seed is idempotent across environments.

---

7. FHIR mapping (forward-compat check)

VitalFlow entity	FHIR R4 resource	Notable mappings
Patient	`Patient`	`given_name/family_name` → `name[].given/family`; `sex_at_birth` → `gender` (partial); `preferred_language` → `communication[].language`
PatientInsurance	`Coverage`	`payer_id` → `payor`; `type` → `type.coding`; `member_id` → `subscriberId`
Appointment	`Appointment`	`start_at/end_at` → `start/end`; `status` → `status` (1:1 enum match); `provider_id` → `participant[].actor`
Encounter	`Encounter`	`class` → `class`; `reason` → `reasonCode`; `location` → `location[].location`

ClinicalNote	`DocumentReference` + `Composition`	SOAP sections map to `Composition.section[]` ; signed state → `DocumentReference.docStatus`
DiagnosisAssignment	`Condition` + `Encounter.diagnosis`	`code` → `Condition.code.coding` ; `rank` → `Encounter.diagnosis[].rank`
ClinicalDocument	`DocumentReference`	`kind` → `type.coding` ; `sha256` + `size_bytes` → `content[].attachment.{hash,size}`

We don't ship FHIR endpoints in V1; the field names above are chosen to make a future adapter a pure projection (no remodel).

---

8. Rollout plan

Phase 1 — ship the TypeScript scaffold (this PR):

- Zod schemas for all 8 entities.
- Service interfaces (no impls).
- Design doc committed.

Phase 2 — UI gap-fill:

- Insurance editor on `/patients/[id]` .
- Diagnosis picker in encounter workspace (new migration for `diagnosis_assignments`).
- Attachment upload UI for `/patients/[id]` and `/encounters/[id]` .

Phase 3 — API surface:

- REST routes under `app/api/v1/*` (Next Route Handlers), each calling the service layer.
- Service implementations backed by `@vitalflow/auth/server` .

Phase 4 — hardening:

- Optimistic locking on note drafts (`updated_at` check).
- Sign/amend RPC so multi-row writes are atomic.
- Diagnosis rank partial unique index.
- Appointment provider-overlap GiST exclusion constraint.